# 2-D Steady-State Heat Conduction in a Rectangular Plate

Point, Line, SOR and ADI Iterative Solvers for the Laplace Equation in C

Course Project 6 – AM5630 Foundation of Computational Fluid Dynamics (Jul–Nov 2024)

Karthick J (OE23S018)
Department of Ocean Engineering, IIT Madras
Faculty: Prof. Arul Prakash

Revised report, October 2026

**Abstract**

The steady temperature field in a $1 \times 2$ plate with a hot bottom wall (100°C) and the other walls at 30°C is computed on a $21 \times 41$ grid with six stationary iterative methods: point Gauss–Seidel (PGS), line Gauss–Seidel (LGS), point and line successive over-relaxation (PSOR, LSOR), and alternating-direction implicit iteration (ADI) without and with relaxation. A review of the original C codes showed that the line and ADI variants never solved a tridiagonal system, that the "ADI with relaxation" update converged to a non-physical fixed point ($T \approx 0$°C in the interior) and that the heat-transfer routine integrated temperature rather than heat flux. After correction (Thomas-algorithm line solves, a consistent relaxation formula and Fourier's-law flux) all six solvers converge to the direct sparse solution of the same discrete system. LSOR and relaxed ADI are the most efficient (69 and 37 iterations at $\omega = 1.5$, 41 and 21 at their optimum $\omega \approx 1.7$–$1.75$), in agreement with the theoretical optimum $\omega_{opt} = 1.78$ for PSOR. The discrete heat input through the bottom wall is $Q = 12.81$ kW per metre depth on the $21 \times 41$ grid and closes the energy balance exactly; it grows logarithmically under refinement because of the temperature discontinuity at the bottom corners, while the interior temperature converges at second order.

## Contents

**1 Problem definition** **3**
1.1 Analytical solution . . . 3

**2 Discretisation** **3**
2.1 Line solves (Thomas algorithm) . . . 4
2.2 Heat transfer through the bottom wall . . . 4
2.3 Algorithm . . . 5

**3 Code review and corrections** **5**

**4 Results** **6**
4.1 Temperature field . . . 6
4.2 Convergence of the iterative schemes . . . 7
4.3 Relaxation factor . . . 8
4.4 Original versus corrected code . . . 8

4.5 Heat balance and grid study . . . . . 8

**5 Conclusions** **9**

**A Corrected source: line SOR** **10**

**B Corrected source: ADI with relaxation (iteration loop)** **13**

# 1 Problem definition

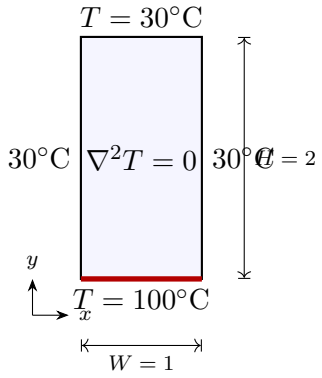



Figure 1: Domain and boundary conditions; $k = 50$ W/m K, unit depth.

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0, \quad 0 < x < 1,\ 0 < y < 2; \qquad T(x,0) = 100,\ T(0,y) = T(1,y) = T(x,2) = 30. \tag{1}$$

Required: (a) the steady temperature contours, (b) the heat transferred across the bottom boundary, (c) the iteration counts of the six schemes. The convergence criterion of the original code, $\max_{i,j} |T_{ij}^{k+1} - T_{ij}^{k}| < 0.01°\mathrm{C}$, is retained. (The original report wrote this as a sum; the code used the maximum, which is what is reported here.)

## 1.1 Analytical solution

With $\theta = (T - 30)/70$, separation of variables gives

$$T(x,y) = 30 + 70 \sum_{n\,\text{odd}} \frac{4}{n\pi} \sin\!\Big(\frac{n\pi x}{W}\Big) \frac{\sinh\big(n\pi(H-y)/W\big)}{\sinh\big(n\pi H/W\big)}, \tag{2}$$

evaluated with 200 odd terms in an overflow-safe exponential form. The bottom corners carry a jump from 100 to 30°C, so $\partial T/\partial y \sim 1/r$ near them and the exact heat flux integrated over the bottom wall is (logarithmically) infinite. Any finite-grid value of $Q$ is therefore grid-dependent (Section 4.5).

# 2 Discretisation

With $N_x = 21$, $N_y = 41$ nodes, $\Delta x = \Delta y = h = 0.05$ and $\beta = \Delta x/\Delta y = 1$, the five-point stencil is

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0, \qquad \tau = \mathcal{O}(h^2), \tag{3}$$

which gives the linear system $A\mathbf{T} = \mathbf{b}$ with $n = 19{\times}39 = 741$ unknowns. $A$ is symmetric positive definite, weakly diagonally dominant and has the block-tridiagonal structure $A = \mathrm{tridiag}(-I,\ B,\ -I)$ with $B = \mathrm{tridiag}(-1, 4, -1)$. Writing $A = D - L - U$ (diagonal, strictly lower, strictly upper parts):

| Scheme | Update | Iteration matrix $G$ |
|---|---|---|
| Jacobi (reference) | $\mathbf{T}^{k+1} = D^{-1}(L+U)\mathbf{T}^k + D^{-1}\mathbf{b}$ | $G_J = D^{-1}(L+U)$ |
| PGS | $(D-L)\mathbf{T}^{k+1} = U\mathbf{T}^k + \mathbf{b}$ | $G_{GS} = (D-L)^{-1}U$ |
| PSOR | $T_{ij}^{k+1} = (1-\omega)T_{ij}^k + \omega\,\tilde{T}_{ij}^{GS}$ | $G_\omega = (D-\omega L)^{-1}[(1-\omega)D + \omega U]$ |
| LGS | block GS, each row solved exactly | $(D_B - L_B)^{-1}U_B$, $D_B = \mathrm{blockdiag}(B)$ |
| LSOR | line solve, then relax with $\omega$ | block SOR |
| ADI | row-implicit sweep, then column-implicit sweep | product of two block-GS operators |

Table 1: Iterative schemes.

For the model problem the Jacobi spectral radius is

$$\rho_J = \tfrac{1}{2}\Big[\cos\frac{\pi}{N_x-1}+\cos\frac{\pi}{N_y-1}\Big] = 0.99230, \qquad \rho_{GS} = \rho_J^2 = 0.9847, \qquad \omega_{opt} = \frac{2}{1+\sqrt{1-\rho_J^2}} = 1.780, \tag{4}$$

and at $\omega_{opt}$ the SOR spectral radius drops to $\omega_{opt} - 1 = 0.78$. The number of iterations needed to reduce the error by a factor $10^{-p}$ is $\approx p\ln 10/(-\ln\rho)$, so PGS should need about twice as many iterations as Jacobi and SOR an order of magnitude fewer.

## 2.1 Line solves (Thomas algorithm)

In LGS/LSOR every interior row $i$ is solved implicitly in $x$ with the neighbouring rows at their latest values:

$$-T_{i,j-1} + 4T_{i,j} - T_{i,j+1} = T_{i-1,j} + T_{i+1,j}, \qquad j = 1,\dots,N_x-2, \tag{5}$$

a tridiagonal system solved in $\mathcal{O}(N_x)$ operations by forward elimination $c'_k = c_k/(b_k - a_k c'_{k-1})$, $d'_k = (d_k - a_k d'_{k-1})/(b_k - a_k c'_{k-1})$ and back substitution $x_k = d'_k - c'_k x_{k+1}$. LSOR then sets $T_{ij} \leftarrow (1-\omega)T_{ij}^{old} + \omega T_{ij}^{*}$. ADI repeats the same in $y$ (columns) after the $x$ sweep, so information crosses the whole domain in both directions in every iteration.

## 2.2 Heat transfer through the bottom wall

By Fourier's law the heat entering through the bottom face (per unit depth) is

$$Q = -k\int_0^W \frac{\partial T}{\partial y}\Big|_{y=0} dx \;\approx\; k\sum_{j=1}^{N_x-2} \frac{T_{1,j}^{wall} - T_{2,j}}{\Delta y}\,\Delta x, \tag{6}$$

which is exactly the flux carried by the five-point stencil across the bottom faces, so the discrete energy balance $Q_{bottom} = Q_{top} + Q_{left} + Q_{right}$ closes to round-off.

### 2.3 Algorithm

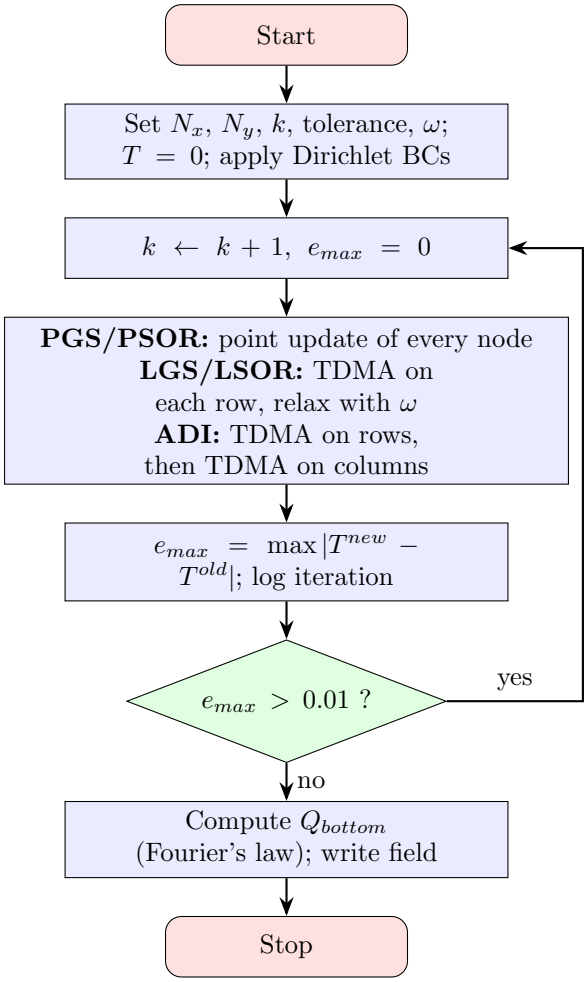



Figure 2: Common structure of the six solvers (`code/*.c`).

## 3 Code review and corrections

| File (original) | Problem | Correction |
|---|---|---|
| all six | `calculateHeatTransfer` returned $k \int T\,dx$ (= 4825 for every scheme). This has units of $\mathrm{W\,m^{-1}K^{-1}}$·K·m but is not a heat rate. | Fourier's-law flux through the bottom faces (Eq. above). |
| `Line_gauss_sidel.c` | Two point-wise sweeps (along $x$, then $y$) per iteration; no implicit line solve. | TDMA on each row (true LGS). |
| `PSOR.c`, `Lsor.c` | Identical programs: two point-SOR sweeps per iteration. Neither is LSOR, and PSOR counted two sweeps as one iteration. | PSOR: one point sweep; LSOR: TDMA row solve + relaxation. |
| `ADI_method.c` | "Iteration level 0.5" used as under-relaxation of point updates; no line solves. | Row TDMA then column TDMA (ADI line iteration). |
| `ADI_method_with_relaxation.c` | $T = T^{old} + \omega \cdot 0.5\,\bar{T}$: the fixed point is $T = 0.5\bar{T}$, i.e. the field is driven towards zero. It "converged" in 8 iterations to $T \approx$ 0–5°C in the interior (Fig. 6). | Row and column TDMA with standard over-relaxation. |

Table 2: Defects and corrections. Originals are in `original/code_JulNov2024/`; the earlier Jan–May 2024 C++ versions are kept in `original/code_JanMay2024/`.

Grid, boundary conditions, initial guess, tolerance, console/file output and code layout were otherwise kept. One routine, `writeFinalField`, was added to write the converged field at full precision for post-processing.

# 4 Results

## 4.1 Temperature field

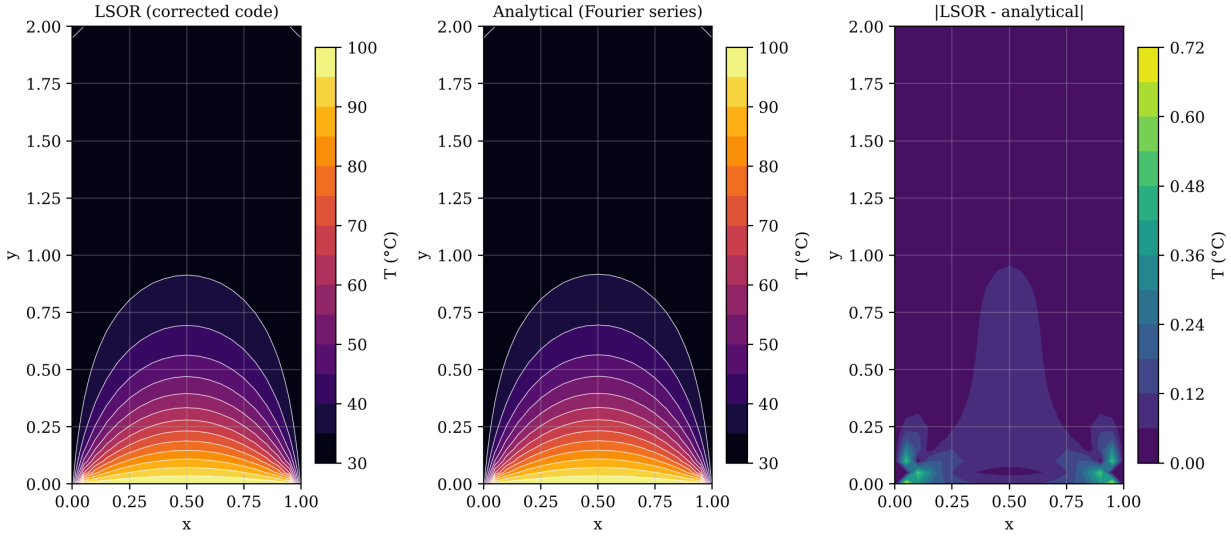



Figure 3: Steady temperature (LSOR, corrected), analytical solution (2) and their difference.

Heat enters through the bottom and leaves mainly through the side walls; the isotherms are symmetric about $x = 0.5$ and decay roughly as $e^{-\pi y}$, so above $y \approx 1$ the plate is within 3°C of 30°C. The numerical and analytical fields agree to better than 0.1°C away from the bottom corners. The largest differences ($\leq$ 0.7°C) sit in the cells next to the corner discontinuities, where the exact solution is not smooth.

### 4.2 Convergence of the iterative schemes

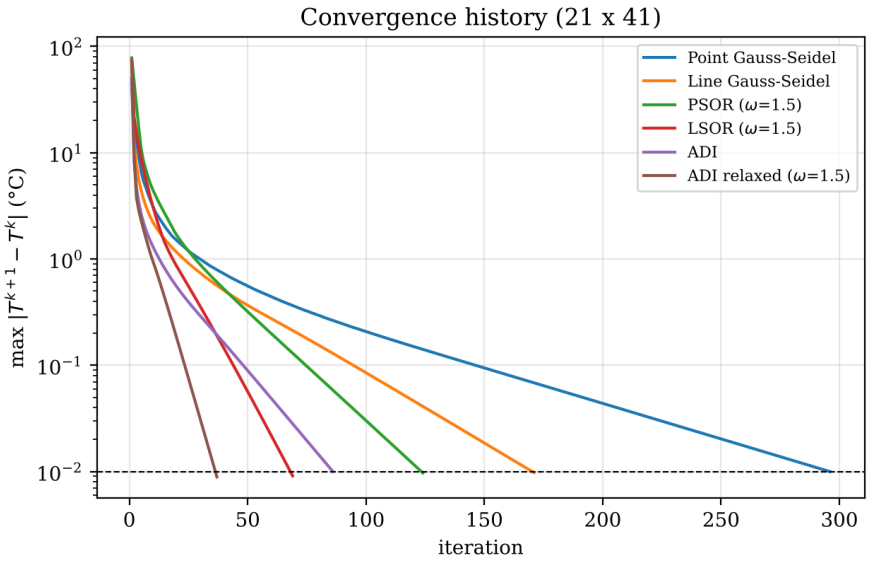



Figure 4: Maximum change per iteration. The straight lines on the log scale are the asymptotic rates $\rho(G)$.

| Scheme | Iterations | $Q_{bottom}$ (W/m) | max $\lvert T - T_{direct}\rvert$ (°C) | Orig. iterations | Orig. "$Q$" |
|---|---|---|---|---|---|
| Point Gauss–Seidel | 296 | 12 842 | 0.64 | 296 | 4 825 |
| Line Gauss–Seidel | 171 | 12 834 | 0.31 | 149* | 4 825 |
| PSOR ($\omega = 1.5$) | 124 | 12 820 | 0.20 | 63* | 4 825 |
| LSOR ($\omega = 1.5$) | 69 | 12 823 | 0.089 | 63* | 4 825 |
| ADI | 86 | 12 831 | 0.31 | 336 | 4 825 |
| ADI relaxed ($\omega = 1.5$) | 37 | 12 819 | 0.081 | 8† | 4 825 |
| Direct sparse solve | – | 12 814 | 0 | – | – |

Table 3: Results on $21 \times 41$. *Two sweeps per counted iteration. †Converged to a wrong field.

Observations:

- The iteration counts follow the theory: line methods couple a whole row implicitly and roughly halve the iterations of their point counterparts (LGS 171 vs PGS 296; LSOR 69 vs PSOR 124), and alternating the line direction (ADI) gains another factor of two.
- The stopping test is on the *increment*, not the error. Since $\|e^k\| \approx \|T^{k+1} - T^k\|/(1-\rho)$, a slowly converging method stops far from the solution: PGS ($\rho \approx 0.985$) stops with an error of 0.64°C even though the increment is $< 0.01$°C. A tolerance of $0.01(1-\rho)$, or a residual-based test, should be used for accuracy-sensitive work.
- The original iteration counts for LGS, PSOR and LSOR look smaller only because each counted iteration contained two sweeps.

### 4.3 Relaxation factor

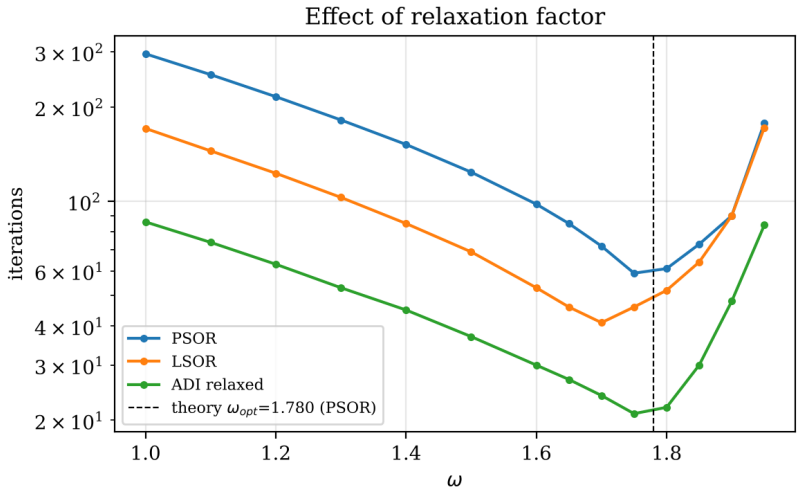



Figure 5: Iterations vs. $\omega$ (corrected PSOR, LSOR and relaxed ADI).

PSOR reaches its minimum (59 iterations) at $\omega = 1.75$, close to the theoretical $\omega_{opt} = 1.78$; LSOR (41 at $\omega = 1.70$) and relaxed ADI (21 at $\omega = 1.75$) have slightly lower optima, consistent with their smaller unrelaxed spectral radii. Beyond the optimum the iteration count rises steeply as $\rho(G_\omega) = \omega - 1$ grows.

### 4.4 Original versus corrected code

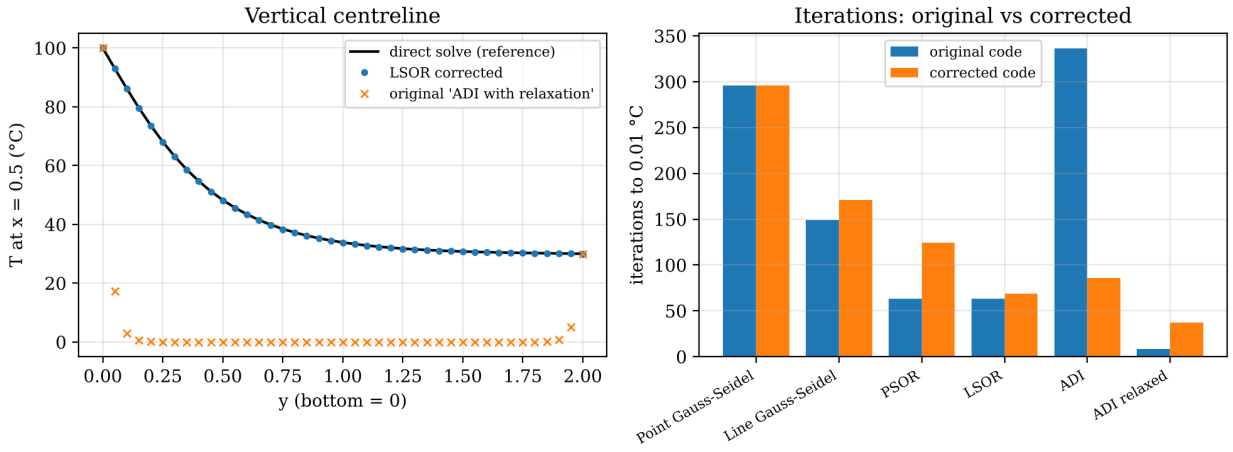



Figure 6: Left: centreline temperature; the original "ADI with relaxation" result collapses to $\approx 0$°C. Right: iteration counts of original and corrected codes.

### 4.5 Heat balance and grid study

For the direct solution on $21 \times 41$: $Q_{bottom} = 12\,813.6$ W/m $= Q_{left} + Q_{right} + Q_{top} = 6\,390.0 + 6\,390.0 + 33.6$ W/m, so the balance closes exactly and more than 99.7% of the heat leaves through the side walls.

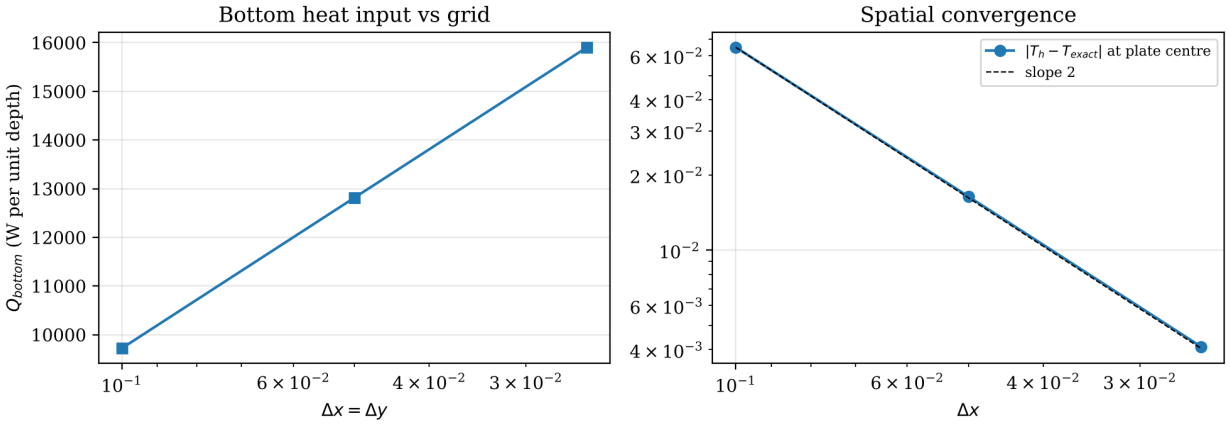



Figure 7: Grid study with LSOR converged to $10^{-9}$: bottom heat input and centre-point error.

| Grid | $Q_{bottom}$ (W/m) | $T(0.5, 1)$ | $\lvert T - T_{exact}\rvert$ |
|---|---|---|---|
| $11 \times 21$ | 9 720 | 33.9067 | $6.5 \times 10^{-2}$ |
| $21 \times 41$ | 12 814 | 33.8583 | $1.6 \times 10^{-2}$ |
| $41 \times 81$ | 15 904 | 33.8461 | $4.2 \times 10^{-3}$ |

Table 4: Grid convergence ($T_{exact}(0.5, 1) = 33.8419$).

The plate-centre error falls by a factor of $\approx 4$ per halving of $h$ (second-order accuracy). $Q_{bottom}$, by contrast, increases by an almost constant $\approx 3.09$ kW/m per halving, i.e. $Q \propto \ln(1/h)$: this is the discrete signature of the $1/r$ flux singularity at the bottom corners. The value 12.8 kW/m is the correct answer *for the* $21 \times 41$ *grid specified in the assignment*, but it is not grid-independent.

# 5 Conclusions

- After correction, all six solvers converge to the direct solution of the discrete Laplace system; the temperature agrees with the analytical series to $< 0.1$°C away from the corners and is second-order accurate.
- Efficiency ranking at $\omega = 1.5$: relaxed ADI (37) < LSOR (69) < ADI (86) < PSOR (124) < LGS (171) < PGS (296). At the optimum $\omega \approx 1.7$–$1.75$ this becomes 21 / 41 / 59 iterations for relaxed ADI / LSOR / PSOR.
- The bottom heat input is $Q = 12.81$ kW per metre depth on $21 \times 41$, with an exactly closed energy balance; it diverges logarithmically with refinement because of the corner temperature jump.
- The original codes contained three method-level errors (no line solves, a scaling factor that drove "ADI with relaxation" to a wrong solution, and a heat-transfer formula without a gradient); these were corrected without changing the problem set-up.

# Reproducing the results

`bash run.sh` compiles and runs the six corrected and six original codes, the $\omega$ sweep and the grid study, then calls `code/postprocess.py` (NumPy, SciPy, Matplotlib).

# A Corrected source: line SOR

```
// 2-D steady heat conduction, Laplace equation on a 1 x 2 plate, 21 x 41 nodes
// Scheme: Line SOR (TDMA along x + over-relaxation)
// Original author: Karthick J (OE23S018). Corrections marked [FIX]/[ADDED].
#include <stdio.h>
#include <math.h>
#include <time.h>

#define M 41 // Number of cells in Y axis (height)
#define N 21 // Number of cells in X axis (width)
#define T_BOTTOM 100.0 // Bottom boundary temperature
#define T_OTHER 30.0 // Other boundaries temperature
#define ERROR_THRESHOLD 0.01 // Convergence criteria
#define K 50.0 // Thermal conductivity (W/mK)

// Temperature grid
double T[M][N];

void initializeGrid()
{
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            T[i][j] = 0; // Set initial temperature to zero
        }
    }
}

// Set boundary conditions
void setBoundaryConditions()
    {
    for (int j = 0; j < N; j++)
    { // Bottom wall (X-axis)
        T[M-1][j] = T_BOTTOM; // Set bottom boundary temperature
    }
    for (int j = 0; j < N; j++)
    { // Top wall (X-axis)
        T[0][j] = T_OTHER; // Set top boundary temperature
    }
    for (int i = 0; i < M; i++)
    { // Left wall (Y-axis)
        T[i][0] = T_OTHER; // Set left boundary temperature
    }
    for (int i = 0; i < M; i++)
    { // Right wall (Y-axis)
        T[i][N-1] = T_OTHER; // Set right boundary temperature
    }
}

// Print grid to file
void printGrid(const char* title, FILE *file)
{
    fprintf(file, "%s:\n", title);
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++)
        {
            fprintf(file, "%5.1f ", T[i][j]);
        }
        fprintf(file, "\n");
    }
}

// Calculate heat transfer along the bottom boundary
// [FIX: the original returned K * integral(T dx), i.e. conductivity x temperature, which is not a heat
// rate. Fourier's law gives q = -K dT/dn. Here the heat entering through the bottom wall (per unit
// depth) is summed over the bottom faces of the first interior row: Q = K * sum (T_wall - T_1)/dy * dx.
// This is the discrete flux used by the 5-point stencil, so the energy balance closes exactly.]
double calculateHeatTransfer()
{
    double total_heat_transfer = 0.0;
    double dx = 1.0 / (N - 1); // Assuming unit length in X direction
```

```
    double dy = 2.0 / (M - 1); // Plate height 2 units in Y direction
    for (int j = 1; j < N - 1; j++)
    {
        total_heat_transfer += (T[M-1][j] - T[M-2][j]) / dy * dx;
    }
    return total_heat_transfer * K; // Multiply by thermal conductivity
}

// [ADDED for post-processing only: full-precision final field, one row of nodes per line]
void writeFinalField(const char* name)
{
    FILE *f = fopen(name, "w");
    for (int i = 0; i < M; i++)
    {
        for (int j = 0; j < N; j++) fprintf(f, "%.10e ", T[i][j]);
        fprintf(f, "\n");
    }
    fclose(f);
}

// [FIX: line methods need an implicit (tridiagonal) solve along each line. The original
// "line" and "ADI" codes only repeated point-wise updates. Thomas algorithm (TDMA):
// a[k] x[k-1] + b[k] x[k] + c[k] x[k+1] = d[k], k = 0..n-1]
void tdma(int n, double *a, double *b, double *c, double *d, double *x)
{
    double cp[256], dp[256];
    cp[0] = c[0] / b[0];
    dp[0] = d[0] / b[0];
    for (int k = 1; k < n; k++)
    {
        double m = b[k] - a[k] * cp[k-1];
        cp[k] = c[k] / m;
        dp[k] = (d[k] - a[k] * dp[k-1]) / m;
    }
    x[n-1] = dp[n-1];
    for (int k = n - 2; k >= 0; k--) x[k] = dp[k] - cp[k] * x[k+1];
}

// Implicit solve along row i (x-direction), neighbours in y taken at their latest values.
// omega = 1 -> line Gauss-Seidel, omega > 1 -> line SOR. Returns the max change on the row.
double rowSweep(int i, double omega)
{
    double a[256], b[256], c[256], d[256], x[256]; int n = N - 2; double err = 0.0;
    for (int k = 0; k < n; k++)
    {
        int j = k + 1;
        a[k] = -1.0; b[k] = 4.0; c[k] = -1.0;
        d[k] = T[i-1][j] + T[i+1][j];
        if (j == 1) d[k] += T[i][0];
        if (j == N - 2) d[k] += T[i][N-1];
    }
    a[0] = 0.0; c[n-1] = 0.0;
    tdma(n, a, b, c, d, x);
    for (int k = 0; k < n; k++)
    {
        int j = k + 1; double old_temp = T[i][j];
        T[i][j] = (1 - omega) * old_temp + omega * x[k];
        if (fabs(T[i][j] - old_temp) > err) err = fabs(T[i][j] - old_temp);
    }
    return err;
}

// Implicit solve along column j (y-direction), neighbours in x taken at their latest values.
double colSweep(int j, double omega)
{
    double a[256], b[256], c[256], d[256], x[256]; int n = M - 2; double err = 0.0;
    for (int k = 0; k < n; k++)
    {
        int i = k + 1;
        a[k] = -1.0; b[k] = 4.0; c[k] = -1.0;
        d[k] = T[i][j-1] + T[i][j+1];
        if (i == 1) d[k] += T[0][j];
        if (i == M - 2) d[k] += T[M-1][j];
    }
    a[0] = 0.0; c[n-1] = 0.0;
```

```
    tdma(n, a, b, c, d, x);
    for (int k = 0; k < n; k++)
    {
        int i = k + 1; double old_temp = T[i][j];
        T[i][j] = (1 - omega) * old_temp + omega * x[k];
        if (fabs(T[i][j] - old_temp) > err) err = fabs(T[i][j] - old_temp);
    }
    return err;
}

int main()
{
    FILE *outputFile;
    outputFile = fopen("temperature_data.txt", "w"); // Open file for writing

    if (outputFile == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }

    // Initialize the grid and set boundary conditions
    initializeGrid();
    setBoundaryConditions();

    printGrid("Initial Temperature Grid", outputFile);

    double max_error;
    clock_t start_time, end_time;

    int iteration_count = 0; // Counter for iterations
    double omega = 1.5; // Relaxation factor

    do {
        start_time = clock(); // Start timer
        max_error = 0.0;

        // Line SOR: TDMA along each row, then over-relaxation of the line solution.
        for (int i = M - 2; i >= 1; i--)
        {
            double error = rowSweep(i, omega);
            if (error > max_error) max_error = error;
        }

        end_time = clock(); // End timer

        iteration_count++; // Increment iteration count

        fprintf(outputFile, "Iteration %d Time: %.6f seconds\n",
               iteration_count,
               (double)(end_time - start_time) / CLOCKS_PER_SEC);

        printGrid("Temperature Grid After Iteration", outputFile);

        printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

    }
     while (max_error > ERROR_THRESHOLD); // Continue until convergence

    printGrid("Final Temperature Grid", outputFile);
    writeFinalField("final_field.dat"); // [ADDED]

    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```

# B Corrected source: ADI with relaxation (iteration loop)

```
    clock_t start_time, end_time;

    int iteration_count = 0; // Counter for iterations
    double omega = 1.5; // Relaxation factor

    do {
        start_time = clock(); // Start timer
        max_error = 0.0;

        // ADI with relaxation: both half-iterations over-relaxed with omega.
        // [FIX: the original update T = (1-w)*T_old + w*0.5*T_new converges to a wrong fixed
        // point T = 0.5*T_new (the factor 0.5 "iteration level" scales the solution), which is
        // why it stopped after 9 iterations with a non-physical field.]
        for (int i = M - 2; i >= 1; i--)
        {
            double error = rowSweep(i, omega);
            if (error > max_error) max_error = error;
        }
        for (int j = 1; j < N - 1; j++)
        {
            double error = colSweep(j, omega);
            if (error > max_error) max_error = error;
        }

        end_time = clock(); // End timer

        iteration_count++; // Increment iteration count

        fprintf(outputFile, "Iteration %d Time: %.6f seconds\n",
                iteration_count,
                (double)(end_time - start_time) / CLOCKS_PER_SEC);

        printGrid("Temperature Grid After Iteration", outputFile);

        printf("Iteration %d Max error: %.6f\n", iteration_count, max_error);

    }
     while (max_error > ERROR_THRESHOLD); // Continue until convergence

    printGrid("Final Temperature Grid", outputFile);
    writeFinalField("final_field.dat"); // [ADDED]

    double heatTransfer = calculateHeatTransfer();
    printf("Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fprintf(outputFile, "Total Heat Transfer along Bottom Boundary: %.2f W\n", heatTransfer);

    fclose(outputFile); // Close the file

    return 0;
}
```